

`production` app — Kapil Enterprises

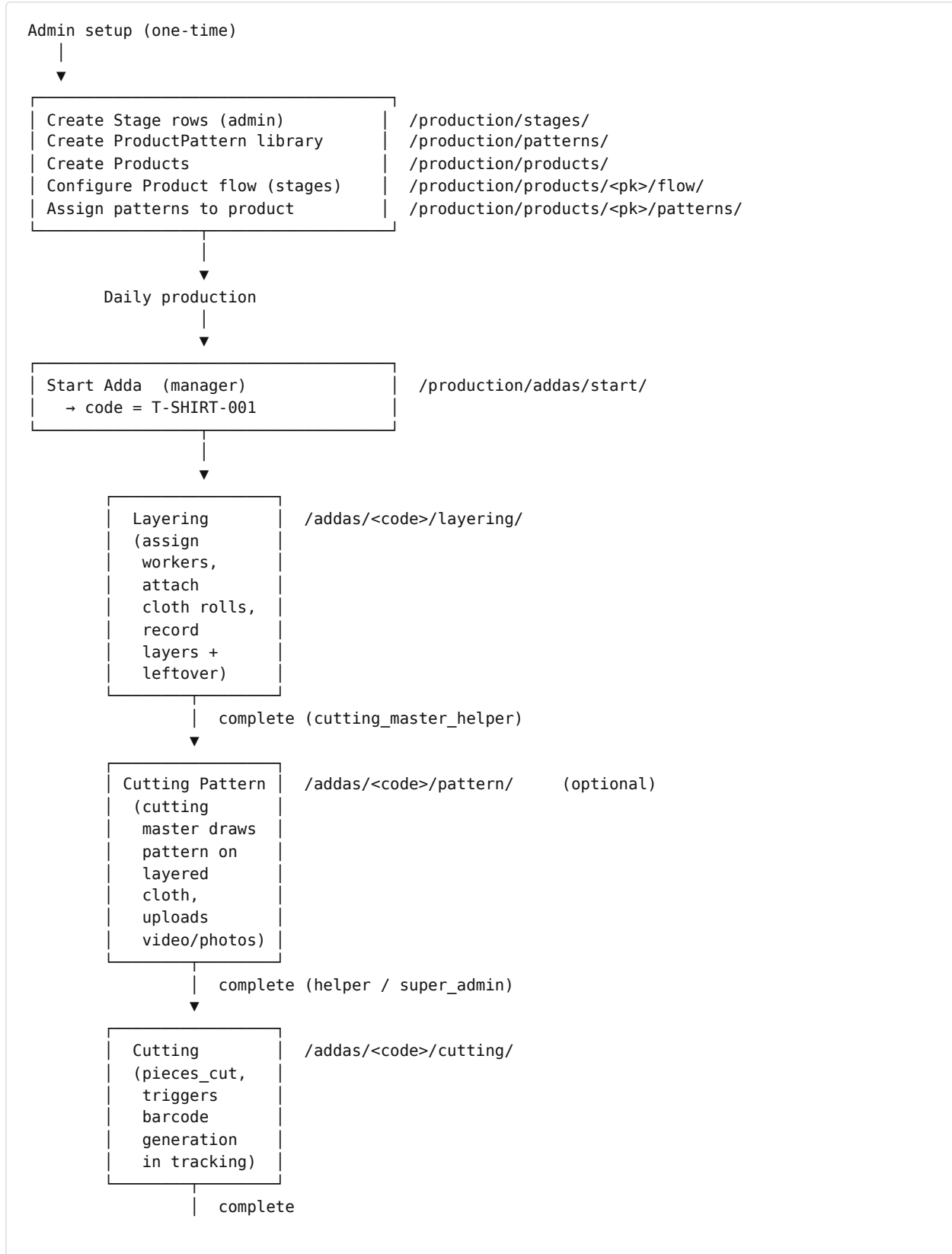
| Junior-Django friendly walkthrough. Hinglish + English.

Yeh app **factory ka production lifecycle** handle karta hai — raw cloth se lekar finished pieces tak. Iss app ki responsibility hai:

- Products define karna (T-SHIRT, NIKKAR, 3-PATTI etc.)
- Reusable **patterns** maintain karna (Front panel, Sleeve etc.)
- **Stages** ka global library (Layering, Cutting Pattern, Cutting...)
- Per-product **workflow** banana (kis order mein stages challenge)
- Live **Adda** (production batch) ka state track karna
- Har stage ka **typed record** save karna (Layering, Cutting Pattern, Cutting)

`raw_materials` upstream hai (cloth inventory) aur `tracking` downstream (barcode + history). Cross-app FK rule: **production app** `raw_materials` se kuch import nahi karta, sirf string FKs use karta hai.

Big picture flow



▼
Adda COMPLETED

Models — ek dum simple explanation

PRODUCT

Factory ka kaunsa item ban raha hai. Example: T-SHIRT.

- `code` (slug, unique) — Adda code is se banta hai (T-SHIRT-001).
- `adda_counter` — kitne Addas iss product ke ab tak ban chuke. Service `SELECT FOR UPDATE` se increment karta hai (race-safe).
- `patterns` (M2M via `ProductPatternAssignment`) — iss product ko kaunse patterns chahiye + kitne pieces.

PRODUCTPATTERN

Pattern library ka reusable shape. Example: “Sleeve”, “Front Panel”.

- `code` (slug) — locked after create kyunki services iss ko refer kar sakte hain.
- `reference_image` — admin sample design upload kar sakta hai.

PRODUCTPATTERNASSIGNMENT (THROUGH TABLE)

Product ↔ ProductPattern jodne wala join row. **Pieces count** carry karta hai.

- Example: T-Shirt = 1 × Front + 1 × Back + 2 × Sleeve → 3 rows.

STAGE

Stages ka master library — admin manage karta hai (`/production/stages/`).

- `code` (slug) — services hardcode karte hain (`STAGE_LAYERING`, `STAGE_CUTTING_PATTERN`, `STAGE_CUTTING`) isliye edit nahi hota.
- `access_by_skill` + `access_by_role` — kaun stage pe kaam kar sakta hai.

WORKFLOWSTAGE

Product → Stage ka join with `order` field. Per-product workflow define karta hai.

- Example: T-Shirt → [Layering(1), CuttingPattern(2), Cutting(3)].

ADDA

Ek production batch. Code = `{Product.code}-NNN`.

- `current_stage` (FK WorkflowStage) — abhi kaunsa stage chal raha hai. `None` = completed.
- `status` — `in_progress` | `on_hold` | `completed` | `cancelled`.

ADDASTAGERECORD

Har Adda + stage combination ka **execution row**. Yahi row workers + started_at + completed_at carry karta hai. Typed records (LayeringRecord, CuttingPatternRecord, CuttingRecord) `OneToOne` se hang karte hain.

LAYERINGRECORD / LAYERINGROLLENTRY / REMAININGCLOTHOFCLOTHROLL

Layering stage ke typed fields.

- `LayeringRollEntry` — har attached roll ka row (width verified, weight verified, layers_on_roll).
- `RemainingClothOfClothRoll` — har entry ka leftover (length + weight).
- `LayeringRecord` — summary (lay_count, layer_length, duration, notes).

CUTTINGPATTERNRECORD + CUTTINGPATTERNPHOTO (NEW)

Cutting-pattern stage ke typed fields.

- `CuttingPatternRecord` — OneToOne stage record + optional video FileField.
- `CuttingPatternPhoto` — multiple photos per record (FK).
- Storage: `media/cutting_pattern/<ADDA_CODE>/video_*. {mp4, ...}` + `photos/*.jpg`. Photos auto-compress via Pillow.

CUTTINGRECORD

Cutting stage ka typed record. `pieces_cut` = denormalized SUM of all `CuttingBundleItem.count` across all bundles (set at complete-time).

CUTTINGPIECEBREAKUP (SECTION 02 — SUGGESTED PIECE BREAKUP / INVENTORY)

Cutting master ka per-(pattern, size, color) inventory row. Acts as available cutting piece inventory:

- `count` = total pieces cut for this combo
- `consumed_count` = pieces moved into bundles (denormalized)
- `available_count` = `count - consumed_count` (computed property)

Pieces flow OUT of this table INTO bundles via `add_pieces_to_bundle`.

CUTTINGBUNDLE + CUTTINGBUNDLEITEM (SECTION 03 — ACTUAL MANUFACTURING BUNDLES)

Bundle = per-size manufacturing container. Items inside = (pattern, color, count).

- `CuttingBundle(cutting_record, size, total_pieces, bundle_number)` — one per (cutting_record, size)
- `CuttingBundleItem(bundle, pattern, color, count, source_breakup)` — items consumed from `CuttingPieceBreakup`

`source_breakup` FK links each item back to the inventory row it came from. On `delete_bundle_item` / `delete_bundle`, the inventory's `consumed_count` is restored automatically.

Bundle drives barcode generation (`generate_for_cutting` iterates items, aggregates by size+color, creates `BarcodeBatch` ranges).

BARCODEBATCH + BATCHBARCODE (DOWNSTREAM — TRACKING APP)

Storage: BarcodeBatch range row per (Adda, bundle, size, color). Lazy per-piece BatchBarcode on first scan. See [TRACKING.md](#).

Stage flow — kaise ek stage agla stage trigger karta hai

1. Worker stage workspace pe details bharta hai (form submit).
2. View ka POST handler **service function** call karta hai (`complete_layering` , `complete_pattern_stage` , `complete_cutting`).
3. Service:
 - Skill / role gate check
 - Typed record save (LayeringRecord / CuttingPatternRecord / CuttingRecord)
 - `advance_to_next_stage(adda, user)` call
4. `advance_to_next_stage` (in `adda_service.py`):
 - Next `WorkflowStage` by order find karta hai
 - `adda.current_stage` update
 - Last stage ho to `adda.status = COMPLETED` , `completed_at` stamp
 - History entry via `tracking.services.log_adda`

Iframe-safe redirect: jab user iframe ke andar Complete dabata hai, view check karta hai

`request.POST['embedded']=='1'` . Agar haan to **new current stage ke embedded panel** pe redirect karta hai with `?advanced=1` . Embedded panel JS `?advanced=1` dekh ke parent ko postMessage bhejta hai: `{type:'stage-advanced'}` . Parent (`adda_detail.html` / `user_dashboard.html`) sun ke `window.location.reload()` chala deta hai.

Roles & responsibilities

Role	Kya kar sakta hai
<code>super_admin</code>	Sab kuch — implicit bypass har perm pe (<code>user_has_perm</code>)
<code>manager</code>	Adda start, workers assign, sab production CRUD
<code>karigar</code>	Stage workspace pe kaam (workers assigned hone par)
<code>accountant</code>	Supplier + Cost Per KG view/edit
<code>listing_team</code>	Storefront app CRUD

Skill-level gating (cutting_master, cutting_master_helper) Stage row's `access_by_skill` M2M se aati hai.

Folder structure

```

config/production/
├─ README.md          ← yeh file
├─ apps.py            ← Django AppConfig
├─ admin.py           ← Django admin registrations
├─ constants.py       ← STAGE_LAYERING, STAGE_CUTTING_PATTERN, STAGE_CUTTING
├─ models.py          ← saare models (Product, Stage, Adda, *Record, *Photo)
├─ forms/             ← Django form classes
│   ├── adda_forms.py
│   ├── product_forms.py
│   └─ layering_forms.py
├─ services/          ← business logic – multi-row writes yahan
│   ├── _shared.py    ← skill/role gate helpers (DRY)
│   ├── adda_service.py ← Adda lifecycle, advance_to_next_stage
│   ├── product_service.py ← Product CRUD
│   ├── layering_service.py ← Layering stage state machine + reopen
│   ├── cutting_pattern_service.py ← Cutting-pattern stage NEW
│   ├── cutting_service.py ← Cutting stage
│   ├── flow_service.py ← per-product Stage list editor
│   ├── access_service.py ← per-stage skill/role gating
│   └─ activity_service.py ← cross-Adda activity timeline
├─ views/             ← Django CBVs + FBVs
│   ├── mixins.py     ← ProductionRoleMixin, SuperAdminOnlyMixin
│   ├── dashboard.py
│   ├── adda_views.py ← Adda list / detail / create
│   ├── product_views.py
│   ├── flow_views.py ← per-product flow editor
│   ├── access_views.py ← Stage library CRUD (perm-gated)
│   ├── pattern_views.py ← ProductPattern CRUD + per-product assign NEW
│   ├── pattern_stage_views.py ← cutting_pattern workspace + actions NEW
│   └─ stage_views.py ← Layering workspace + stage panel dispatcher
├─ templates/production/ ← page-scoped HTML
│   ├── adda_detail.html ← Adda detail (tabbed pipeline)
│   ├── adda_list.html, adda_form.html
│   ├── product_list.html, product_form.html, product_flow.html
│   ├── product_patterns_edit.html ← NEW
│   ├── pattern_list.html, pattern_form.html, pattern_confirm_delete.html ← NEW
│   ├── stage_list.html, stage_form.html, stage_confirm_delete.html
│   ├── layering_workspace.html, _stage_panel_layering.html
│   ├── pattern_workspace.html, _stage_panel_cutting_pattern.html ← NEW
│   ├── cutting_form.html, _stage_panel_cutting.html
│   ├── stage_panel_embedded.html ← iframe-friendly wrapper
│   └─ _form_styles.html, _activity_feed.html, ...
├─ urls.py            ← route table
├─ migrations/        ← schema changes (0001 → 0014)
└─ tests/             ← pytest-style tests

```

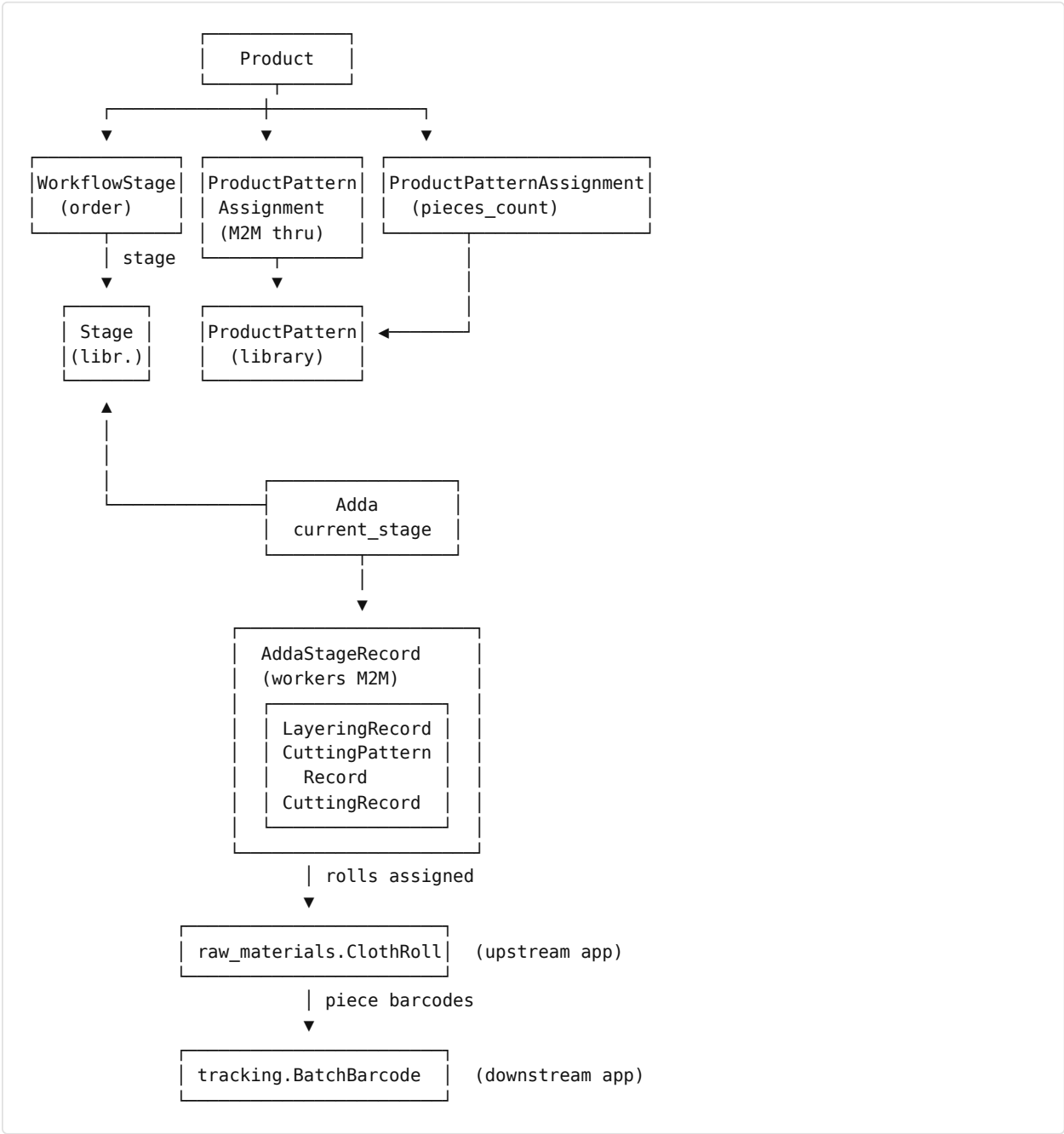
Important Django concepts in this app

Concept	Where to see
Service layer owns multi-row writes (CLAUDE.md rule #4)	<code>services/*.py</code>
<code>@transaction.atomic</code> wraps every mutating service	e.g. <code>complete_layering</code> , <code>reopen_layering</code>
<code>select_for_update</code> for race-safe counters	<code>adda_service.create_adda</code>
Permissions via <code>permission_service</code> — no raw <code>is_superuser</code>	view mixins + <code>user_has_perm</code>
Iframe + postMessage for inline stage panels	<code>stage_panel_embedded.html</code> + <code>adda_detail.html</code>
Pillow image compression on save	<code>cutting_pattern_service._compress_image</code>
<code>STORAGES['default']</code> abstraction for media (S3-swappable)	<code>config/config/settings/base.py</code>
Generic FileField/ImageField <code>upload_to=callable</code>	<code>_cutting_pattern_video_path</code> etc.

Request lifecycle (example: “Complete Layering” click)

1. Browser: POST /production/addas/T-SHIRT-001/layering/complete/
(form submit from iframe, embedded=1, csrf_token included)
2. urls.py routes → LayeringCompleteView (production/views/stage_views.py)
3. LoginRequiredMixin → user logged in?
ProductionRoleMixin → user in PRODUCTION_ROLES?
4. View.post():
 - request.POST parse → action ('draft' | 'complete'), header values, per-entry breakup data
 - save_layering_draft(...) – lax write of header + per-entry
 - if action=='complete':
 - complete_layering(...) – strict service call
 - ├─ @transaction.atomic
 - ├─ _ensure_can_manage(user)
 - ├─ _ensure_can_complete_layering(user)
 - ├─ validate: duration > 0, layer_length > 0, all entries have layers + leftover
 - ├─ per-roll layers_on_roll stamp onto ClothRoll
 - ├─ LayeringRecord.objects.create(...)
 - └─ advance_to_next_stage(adda, user)
 - ├─ find next WorkflowStage by order
 - ├─ update adda.current_stage (or COMPLETED if last)
 - └─ log_adda(AddaHistory.STAGE_ADVANCED)
5. View returns 302 redirect:
 - if embedded=1: → /production/addas/<code>/stage/<next>/?embedded=1&advanced=1
 - else: → /production/addas/<code>/
6. Browser follows redirect inside iframe → new stage panel loads.
JS reads ?advanced=1 → postMessage parent {type:'stage-advanced'}.
Parent listens → window.location.reload() → fresh state shown.

Database relations diagram



Step-by-step: T-SHIRT-001 ka complete journey

Step	Who	Action	Where
1	Manager	Start Adda for T-SHIRT product	<code>/production/addas/start/</code>
2	(system)	Adda code = <code>T-SHIRT-001</code> , current_stage = Layering wf	<code>create_adda</code> service
3	Manager	Assign workers (cutting_master + helper)	<code>/addas/T-SHIRT-001/layering/</code> Section 02
4	Master	Attach cloth rolls + verify width/weight	Section 02 attach form
5	Master	Fill per-roll layers + leftover breakup	Section 04 form
6	Helper	Click “Complete Layering → Advance”	Section 04 button
7	(system)	LayeringRecord created → advance → current = Cutting Pattern	<code>complete_layering</code> service
8	Master	Open <code>/addas/T-SHIRT-001/pattern/</code> — see pattern checklist	Section 01
9	Master	Pick video file (auto-submits)	Section 03
10	Master	Pick photos (auto-submits each batch)	Section 04
11	Helper	Click “Complete Pattern” → confirm	Section 05
12	(system)	Advance → current = Cutting	<code>complete_pattern_stage</code>
13	Manager	Open Cutting workspace <code>/addas/<code>/cutting/workspace/</code> — Section 02 inventory + Section 03 bundles	<code>cutting-workspace</code>
14	Master	Fill Section 02 cutting pieces (Pattern × Size × Color × Count)	<code>upsert_breakup_row</code>
15	Master	Section 03 “+ Create New Bundle” — pick size + multi-select Pattern × Color × Take from inventory	<code>create_bundle_with_pieces</code> (atomic header + items)
16	(system)	<code>CuttingBundleItem</code> rows created; <code>consumed_count</code> incremented on source breakup rows	service
17	Helper	Section 05 — review barcode preview (Bundle × Color × Start..End ranges) → Complete Cutting	<code>complete_cutting</code>
18	(system)	<code>BarcodeBatch</code> ranges created (one per bundle × color); <code>BatchBarcode</code> lazy on scan	<code>tracking.services.generate_for_cutting</code>

Step	Who	Action	Where
19	(system)	Adda status = COMPLETED (if last stage), current_stage = None	<code>advance_to_next_stage</code>

Admin tools (out-of-band corrections)

- **Layering Reopen** (admin only): `POST /addas/<code>/layering/reopen/` — completed Layering ko unlock karta hai correction ke liye. Header values (length/duration/notes) `sr.draft_*` mein wapas copy hote hain. Refuses if downstream stage already started.
- **Stage library editor**: `/production/stages/` — perm-gated (`production.change_stage`). Super Admin bypass.
- **Per-product flow editor**: `/production/products/<pk>/flow/` — drag stages in/out of product workflow.

Tests + verify

```
cd /home/tech/umesh-personal/django_inventory
env/bin/python config/manage.py check #
clean
env/bin/python config/manage.py test raw_materials production tracking inventory accounts #
74/74
env/bin/python config/manage.py runserver
```

Where to read next

- `../docs/production/OVERVIEW.md` — subsystem index
- `../docs/production/CUTTING_PATTERN.md` — cutting_pattern stage deep dive
- `../docs/production/LAYERING_STAGE.md` — layering stage deep dive
- `../docs/archive/production/CHAT_LOG.md` — chronological design decisions (*archived*)
- `../CLAUDE.md` — repo-wide rules (service layer, RBAC, no signals)

2026-06-12 status + data-flow card (V2 era — read this first)

Truth model (LIVE since V2-1 → V2-3): `WorkerStageTask` = participation truth (partial-unique active per (stage_record, worker); cancel-not-delete) · `WorkerStageContribution` = PRODUCTION truth (`reported_quantity` immutable; `verified_quantity` = management correction via the review page; frozen `expected_*` = visibility, never money). Canonical: [docs/ARCHITECTURE_V2.md](#).

```
Adda start → stage records (one per WorkflowStage) → workers assigned
(set_stage_workers – SOLE WST writer, CI gate [4/4])
→ worker reports on phone (report_contributions → WSC lines)
→ complete_worker_task FREEZES expected_rate/_earning
→ stage completes (unreported tasks auto-cancel – F3; P2 dialog warns first)
→ settlement reads verified-else-reported (expense app's job)
```

Single-writer rules here: WST/WSC → `worker_task_service` only (C-TM: EVERY capture path — manual today, barcode later — converges through it) · stage records/typed records → each stage's service via the shared skeletons · costing freeze → `cost_service`.

What breaks if bypassed: a WSC written elsewhere skips the expected-* freeze and the settlement guards — a payable line that money code can't trust; a second WST writer breaks the roster=participation promise.

Mobile-first (rule 11): worker report flow is phone-first (verified 360px); every management table needs an explicit responsive strategy.

Django Learning Notes (V2 additions)

- **Partial unique constraint** (`UniqueConstraint(condition=~Q(status='cancelled'))`): one ACTIVE task per worker-stage while letting cancelled history pile up.
- **Frozen snapshots:** write-once columns set inside the completing transaction (price-at-time-of-order pattern) — later rate edits change nothing retroactively.
- **Open-closed stage engine:** `StageHandler.contribution_schema(adda)` — the worker form renders from a schema; new stage = new handler, zero UI edits.
- **Template-method reopen** (`_shared.reopen_stage_record`): shared skeleton
 - per-stage guard/teardown; V2-3 armor blocks reopen when settlement money exists (names the ADST).